

實驗紀錄 v1

0. 研究範圍

本專案是研究單一 App 越權風險模型，分析對象是單一 APK 內部的元件宣告與其互動關係，相較於 El-Zawawy 與 Hamdy (2025) 針對多個應用程式之間所提出之完整跨 App 的 n-order privilege escalation chain 偵測方法，本研究借鑒其路徑分析的邏輯，應用於單一應用程式內部的元件互動分析。

本專案的機器學習模型定位為輔助工具，規則引擎 (privilege_rules.py) 提供判斷依據，ML 模型提供量化風險評分，規則引擎與模型的判斷結果統一顯示在報告中，並且模型不會取代規則引擎的判斷邏輯。

1. 資料來源

1.1 Toy APK (人工設計, 30 個)

依照 toy_apk_scenario_design.md 的規劃，設計 5 個威脅分類，每個分類各有 6 個設計案例 (1 個對照組 + 5 個變體)，label_source=manual

1.2 真實資料 (MalDroid 2020, 38 個)

使用 **MalDroid 2020** 做為來源，其本身已涵蓋良性與惡意類別的完整 APK 檔案，可簡化蒐集流程。合計 30 (toy) + 38 (Benign 18 / Banking 10 / SMS 10) = 68 個 APK，本階段主要聚焦於驗證架構的最小可行性，因此實際使用的 APK 數量遠低於 PLAN.md Task 5 訂定的 100 ~ 150 的硬性上限。

1.3 Dataset 範圍

Task 8 第 3 點原本列出的 5 項場景中 (正常 / 過度越權 / exported unprotected service / provider leak / receiver broadcast theft)，provider leak 對應場景 E (IPC_PROVIDER_REDELEGATION)，缺少場景 C (IPC_CONFUSED_DEPUTY)。最終決定沿用 Task 5 已完成的 Scenario A-E (30 個 toy APK，每個場景 6 個案例，label_source=manual)，正常對照組取自各場景的 label = 0 案例。

2. 標記方式

2.1 Label 定義

training_data_spec.md §2.1 中，定義 filter_row label 公式：

```
label = 1 if (exported == True and protected == False) else 0
```

- Toy 資料：

label_source=manual，最初設計時決定每個元件是否構成風險，作為驗證規則引擎和特徵抽取正確性的基準

- 真實資料：

- `label_source=rule_weak_label`，由 `privilege_rules.py` 輸出弱標註，另外用 `check_launcher_pattern.py` 系統性掃描初篩後，針對三類代表性樣本（第三方 SDK / App 自身非-launcher 進入點 / banking 類別）各挑選案例，以 `audit_apk.py` 深度人工復核（複核範圍見 §5.4）

2.2 APK 層級資料切分

`split_dataset.py` 以 APK 為單位，對資料集進行切分，分成 70%/15%/15% train/val/test，避免同一 APK 的多個元件同時出現在訓練集與測試集造成資料洩漏。首次未加分層抽樣時，出現 test split 負樣本僅 1 筆的極端分布，改用 `--stratify`（依 label 分層抽樣）後，三個 split 的正負比例趨近整體資料集。

展開後共 256 筆 `filter_row` 訓練樣本：

Split	樣本數	正樣本 (label=1)	負樣本 (label=0)
train	184	127	57
val	31	23	8
test	41	30	11

3. 模型版本

3.1 `filter_row`（已訓練）

項目	內容
模型	RandomForestClassifier (scikit-learn)
超參數	<code>n_estimators=200</code> 、 <code>class_weight="balanced"</code> 、 <code>random_state=42</code>
特徵維度	109 (categorical one-hot + boolean + multi-hot action/category/data_type)
Encoder	FilterRowEncoder：categorical 欄位用 OneHotEncoder(<code>handle_unknown="ignore"</code>)，multi-hot 欄位用 MultiLabelBinarizer，缺值統一轉換成 "<NONE>"
選擇理由	multi-hot 特徵稀疏， <code>feature_importances_</code> 支援可解釋性報告，資料不易 overfit

本階段目標是驗證 pipeline 架構跑得通、feature importance 是否符合直覺，而非追求最高準確率，因此未做大規模 hyperparameter search，而是只用合理預設值。

3.2 `resolution_row`（v2 階段，尚未實作）

`resolution_row` 特徵產生已於 `_build_resolution_rows()`

（`parse_manifest.py:257`）實作，計算 `match_action/match_category/`

match_type、caller/callee permission 與 risk_hint。feature_schema.json 也已預留 boolean/categorical 欄位定義並註記 "status": "pending_v2"。但目前 encoder.py 內只有 FilterRowEncoder、build_training_data.py 只輸出 row_type: "filter"、trainer.py 只訓練 RandomForestClassifier，dataset/training 底下也只有 filter_rows.jsonl 與 model_v1 成品。v1 階段完成特徵層與 schema 層的預留，需等到 bytecode v2 補強真實 sender 資料才會實作並訓練。

3.3 intent_row (不訓練)

intent_row 並非獨立的訓練目標，是作為中繼特徵表示，與 filter_row 配對後產生 resolution_row 的 match_action/match_category/match_type 等欄位。v1 manifest-only 階段，intent_row 是由 filter_row 以 1:1 方式反推而來，sender 固定為 <UNKNOWN>，因此配對結果不具鑑別力，等到 v2 bytecode 補強後，intent_row 將改為真實的元件互動資料（真實 sender、真實發送的动作），但 intent_row 本身仍不會被單獨訓練。

4. Metrics (filter_row, v1)

4.1 整體結果

Split	n	Precision	Recall	F1	Accuracy
train	184	1.0	1.0	1.0	1.0
val	31	1.0	1.0	1.0	1.0
test	41	1.0	1.0	1.0	1.0

三個 split 的 Confusion matrix ($\begin{matrix} TN & FP \\ FN & TP \end{matrix}$) 分別為 train $\begin{matrix} 57 & 0 \\ 0 & 127 \end{matrix}$

val $\begin{matrix} 8 & 0 \\ 0 & 23 \end{matrix}$ test $\begin{matrix} 11 & 0 \\ 0 & 30 \end{matrix}$ ，均無任何誤判。

4.2 Feature Importance (前 3 名)

permission_<NONE> > protected > exported，與 label 的定義邏輯一致，也驗證 encoder／訓練流程合理。

4.3 實驗結果

本次實驗中重要的已知限制：label 公式本身即為 exported and not protected 的直接函數，而 exported／protected 同時也是模型的輸入特徵，因此在設計上模型結果必然可達到 100%完美準確率。然而這不代表模型具備超越規則引擎的判斷能力，只是能夠驗證 v1 階段「encoder 正確轉換特徵、訓練流程正確、pipeline 架構跑得通」所要求的目標。

5. 已知限制

5.1 filter_row 特徵與 label 的資料洩漏

如 §4.3 所述，`exported` / `protected` 這兩個屬性同時出現在 `features` 與 `label` 的定義中，且 `label` 的函數是這兩個特徵的確定性函數，代表 `label` 與特徵之間為重言式（tautological）關係，這也顯示本模型弱監督（weak supervision）架構的特性。

5.2 IPC_PROVIDER_URI_GRANT_BYPASS 訓練樣本代表性限制

此 finding 發生在 `protected == True` 的情況（有 `read/write permission` 保護，但 `grantUriPermissions=true` 可能繞過保護）。在目前的 `filter_row` `label` 公式下，這類的 `row` 的 `label` 永遠為 0，即使規則引擎正確標記出風險，此 `pattern` 也不會成為訓練資料中的正樣本。

5.3 top_features 為全域排名，非 per-instance 解釋

`RandomForestClassifier` 僅提供全域 `feature_importances_`，不提供 per-instance 的 local explanation。從實際推論的輸出中可以觀察到：同一批 component-level predictions（7 筆 `row`）的 `top_features` 完全相同（`permission_<NONE>` > `protected` > `exported` > `permission_DUMP` > `has_action_androidx.profileinstaller...`），即為此限制在真實輸出中的具體樣貌。

5.4 IPC_CONFUSED_DEPUTY 對真實世界 App 的語意落差

5.4.1 發現過程

蒐集完真實世界資料後，38 筆中有 18 個 benign（MalDroid 2020 官方標記為良性）樣本，在 `extractor bug` 修復後，全都被標記 `label=1`（100%）。使用 `check_launcher_pattern.py` 進行系統性掃描後，發現有 9 筆的觸發原因是 `MAIN/LAUNCHER` 進入點，剩下的 29 筆含有非-launcher 的 `exported` 元件，也間接推翻「規則只是誤判 launcher」的簡化假設。

5.4.2 四類 exported 元件性質

進一步以 `audit_apk.py` 深度核對 7 筆真實世界樣本後，確認「`exported` 且無 `permission` 保護」的元件，在安全意義上可拆分成四類不同的性質：

類型	例子	攻擊者能塞什麼進去？
① 純 launcher 進入點	每個 App 的 <code>MainActivity</code> ， <code>intent-filter</code> 只有 <code>MAIN/LAUNCHER</code>	什麼都塞不了，不帶任何資料欄

		位，系統只用它 啟動 App 本身
② 第三方 SDK 官方要求的宣告	Facebook SDK 的 FBUnityDeepLinkingActivity、Unity 的 UnityPlayerActivity、Firebase 的 FirebaseInstanceIdService	開發者沒有選擇空間，是 SDK 官方整合文件要求的固定寫法
③ Android 平台強制要求	App Widget 的 APPWIDGET_CONFIGURE/_UPDATE、MediaRouteProviderService、媒體按鍵 receiver	OS 層級要求 exported 才能正常運作，不是開發者的安全疏失
④ 真正自訂、可能有意義的進入點	接受自訂 URI scheme 的深連結 Activity（如 neuronation://）、自訂 broadcast action 的 receiver	攻擊者可構造 Intent、傳入自訂資料

目前 EXPORTED_UNPROTECTED_ACTIVITY/_RECEIVER 與 IPC_CONFUSED_DEPUTY 規則不區分這四類，只要「元件 exported 且無 permission」+「App 持有任一危險權限」就會觸發。

5.4.3 人工抽查案例分析（7 筆）

#	樣本 (package)	類別	判斷	理由摘要
1	org.android.system	banking	同意	.SMSReceiver 同時監聽 SMS_RECEIVED+NEW_OUTGOING_CALL， READ_SMS/SEND_SMS/WRITE_SMS/INTERNET， package 偽裝系統名稱——第④類，簡訊攔截/OTP 竊取教科書手法
2	net.marlove.mockgps	benign (對照組)	不同意	MainActivity 僅純 MAIN/LAUNCHER (第①類)， IPC_CONFUSED_DEPUTY 純因 App 持有定位權限觸發，而定位權限是「模擬 GPS」工具的核心功能本身
3	com.hdgames.superbrosadve	benign (第三方)	不同意	被標記元件全部是 Unity

	ntures	SDK)		引擎與 Facebook SDK (第②類) 官方文件要求的標準宣告
4	air.nn.mobile.app.main	benign (第三方 SDK)	部分同意	.AppEntry 額外接受自訂 scheme 深連結 (第④類, 真正有意義), 但同一 finding 混入 Amazon IAP/Adjust/Firebase SDK 元件 (第②類), 稀釋訊號辨識度
5	com.shoushou88.zhezhe99	banking	同意	SMSReceiver+PhoneListene r+BootBroadcastReceiver (含疑似 C2 回報 action) + 23 個危險權 限, 多重訊號疊加, 第④ 類, 銀行木馬特徵完整
6	com.sonos.acr	benign (非- launcher)	不同意	被標記元件為 widget 設 定/更新、 MediaRouteProviderService ——全部屬第③類, Android 平台機制強制要 求
7	com.reneph.passwordsafe	benign (非- launcher)	不同意	僅 launcher (第①類) 與 標準 widget config (第③ 類) 被標記, 權限清單無 任何個資類危險權限, IPC_CONFUSED_DEPUT Y 純因持有 WRITE_EXTERNAL_STO RAGE (備份匯出合理用 途) 觸發

判斷的那一欄, 三個判定類別指的是人工使用 audit_apk.py 核對規則引擎 (privilege_rules.py) 輸出的 finding 後, 判斷「規則觸發了, 但這個觸發在資安意義上到底有沒有道理」。

7 筆的整體結果是 2/7 同意 (皆為 banking)、4/7 不同意 (皆為 benign)、1/7 部分同意。其中兩個判斷正確的案例, 共通點是多重訊號疊加、且觸發元件: 全部屬於第④類, 四個誤判案例, 共通點是觸發元件屬於第①②③類, 且往往只靠單一 finding 就把

整個 App 判定為高風險。這也顯示出目前 IPC_CONFUSED_DEPUTY 規則對真實世界 App 的元件複雜度門檻過寬，造成 benign 樣本大量誤標為 label=1，而非規則引擎本身邏輯失效。

5.4.4 結論與處理方式

規則引擎對 manifest 的擷取（exported/permission/ intent-filter）每一筆皆正確，然而問題出在判斷真實世界 App 的元件時，IPC_CONFUSED_DEPUTY 的觸發門檻過於寬鬆，無法進一步區分四類 exported 元件性質。目前 v1 階段暫不修改規則邏輯，單純以文件明確揭露此限制，修改規則、增加 intent-filter action 判斷與已知平台機制 allowlist 可延後至 v2 階段。

6. 可重現性測試

測試範疇	檔案	案例數	狀態
Vector schema／feature builder	test_parse_manifest.py	25	passed
Encoder（unknown category／missing permission）	test_encoder.py	10	passed
Pipeline 整合（模型存在/不存在兩種情境）	test_pipeline_integration.py	13	passed
完整套件	—	166	passed

app.main subprocess 介面對真實 APK 的收尾驗證（sha256 0c98cca2...）：
app_risk_probability=1.0，8 筆 component-level predictions，格式吻合 inference_data_spec.md §2。規則引擎的 findings 與模型 ML_RISK_ASSESSMENT 並存於同一份報告。缺少模型情境下，report.status 維持 success，並且觸發 MODEL_NOT_AVAILABLE，驗證平台在沒有模型時仍可提供基本靜態分析。